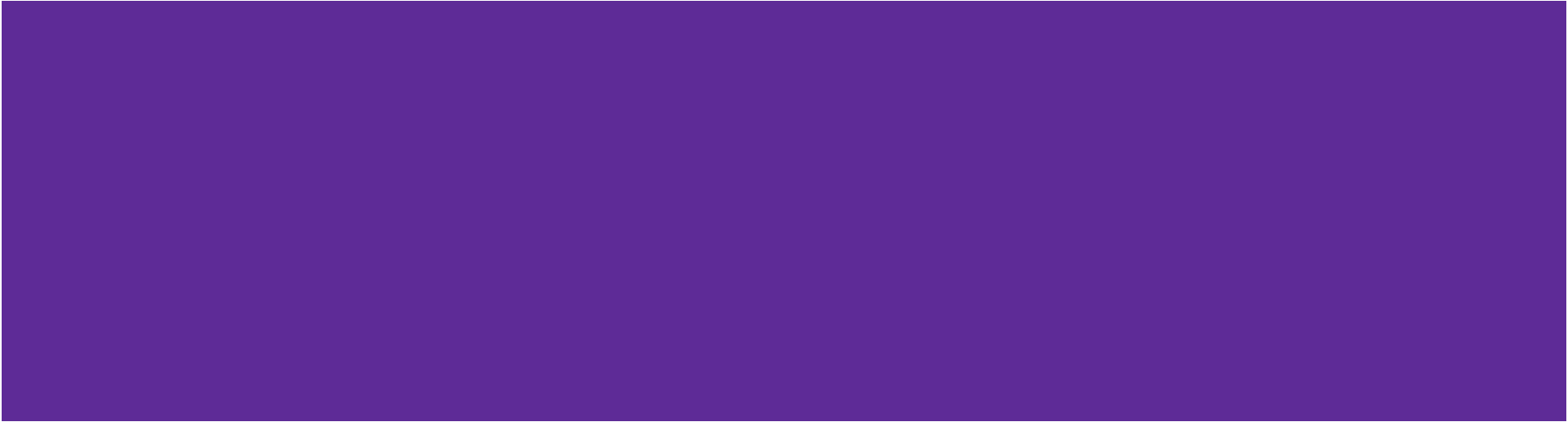


DISTRIBUTED AND CLOUD COMPUTING

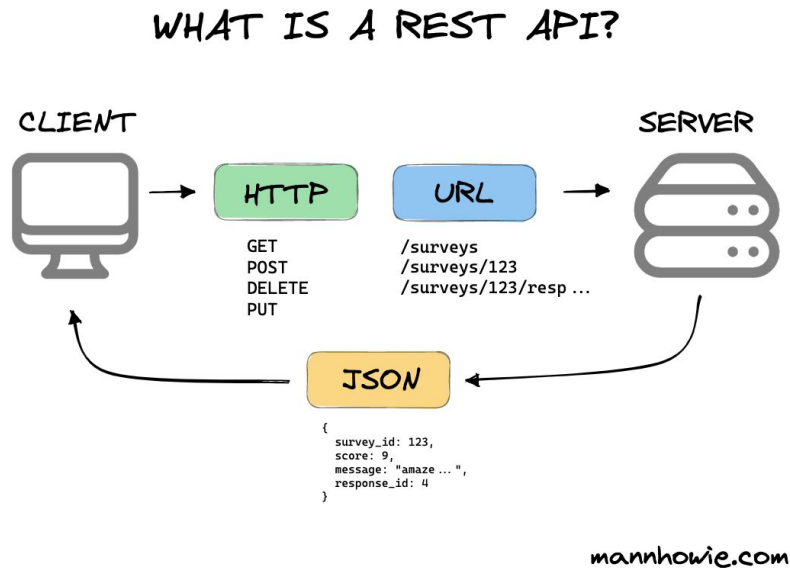
LAB 7: RESTFUL API & OPENAPI SPECIFICATION

(Module: Services & API Architectures)



REST As an Architecture Style

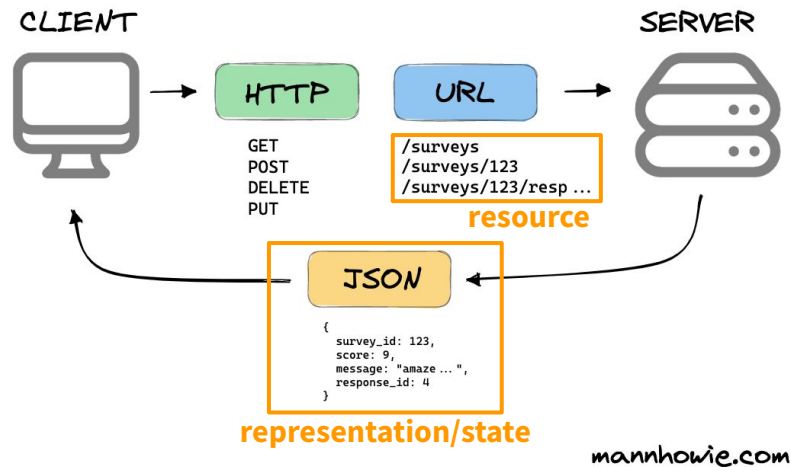
- REpresentational State Transfer: architecture style for distributed hypermedia systems
- REST Components
 - a. Resource
 - Identifiers (URI including URL)
 - Metadata (e.g., source links, etc.)
 - b. Representation
 - Metadata (e.g., media type, etc.)
 - c. Control data (e.g., Cache-Control, etc.)
- 6 Guiding Principles/Constraints of REST
 - a. Client-Server
 - b. Stateless**
 - c. Cacheable
 - d. Uniform Interface
 - e. Layered System
 - f. Code on Demand (Optional)
- **RESTful API (or REST API)**: Web API conforming to the REST architecture style



REST Components

- Resource: **abstraction** of information
 - Identifiers
 - URL endpoint...
 - Metadata
 - Source links
 - Alternate data formats
 - Vary: language preferences...
 - ...
- Representation: **state** of the resource
 - Metadata
 - Media type
 - Last-modified time
 - ...
- Control data: interaction behavior
 - Cache-Control
 - ...

WHAT IS A REST API?



REST Components

- Resource: **abstraction** of information
 - Identifiers
 - URL endpoint...
 - Metadata
 - Source links
 - Alternate data formats
 - Vary: language preferences...
 - ...
- Representation: **state** of the resource
 - Metadata
 - Media type
 - Last-modified time
 - ...
- Control data: interaction behavior
 - Cache-Control
 - ...

```
{
  "id": 123,
  "title": "What is REST",
  "content": "REST is an architectural style for building web services...",
  "published_at": "2023-11-04T14:30:00Z",
  "author": {
    "id": 456,
    "name": "John Doe",
    "profile_url": "https://example.com/authors/456"
  },
  "comments": {
    "count": 5,
    "comments_url": "https://example.com/posts/123/comments"
  },
  "self": {
    "link": "https://example.com/posts/123"
  }
}
```

Pressing the links (i.e., GET ...) transform from one resource state to another - REST.

REST Components

- Resource: **abstraction** of information
 - Identifiers
 - URL endpoint...
 - Metadata
 - Source links → **JSON, XML, ...**
 - Alternate data formats
 - Vary: language preferences...
 - ...
- Representation: **state** of the resource
 - Metadata
 - Media type → **application/json**
 - Last-modified time
 - ...
- Control data: interaction behavior
 - Cache-Control
 - ...

X	Headers	Preview	Response	Initiator	Timing
▼ General					
Request URL	https://raw.githubusercontent.com/gin-gonic/gin/refs/heads/master/AUTHORS.md				
Request Method	GET				
Status Code	● 304 Not Modified				
Remote Address	127.0.0.1:10808				
Referrer Policy	no-referrer-when-downgrade				
▼ Response Headers					
Access-Control-Allow-Origin	*				
Cache-Control	max-age=300				
Cross-Origin-Resource-Policy	cross-origin				
Date	Fri, 24 Oct 2025 14:38:56 GMT				
Etag	W/"90f9c0f1663fa52846bcbe86835fb5a3346653e67a26c6fee0fe2635c357e290"				
Expires	Fri, 24 Oct 2025 14:43:56 GMT				
Vary	Authorization,Accept-Encoding				
▼ Request Headers					
:authority	raw.githubusercontent.com				
:method	GET				
:path	/gin-gonic/gin/refs/heads/master/AUTHORS.md				
:scheme	https				
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image				
Accept-Encoding	gzip, deflate, br, zstd				
Accept-Language	en-US,en;q=0.9				
Cache-Control	max-age=0				
If-None-Match	W/"90f9c0f1663fa52846bcbe86835fb5a3346653e67a26c6fee0fe2635c357e290"				

GET <https://raw.githubusercontent.com/gin-gonic/gin/refs/heads/master/AUTHORS.md>

REST Components

- Resource: **abstraction** of information
 - Identifiers
 - URL endpoint...
 - Metadata
 - Source links
 - Alternate data formats
 - Vary: language preferences...
 - ...
- Representation: **state** of the resource
 - Metadata
 - Media type
 - Last-modified time
 - ...
- Control data: interaction behavior
 - Cache-Control
 - ...

X Headers Preview Response Initiator Timing	
▼ General	
Request URL	https://raw.githubusercontent.com/gin-gonic/gin/refs/heads/master/AUTHORS.md
Request Method	GET
Status Code	● 304 Not Modified
Remote Address	127.0.0.1:10808
Referrer Policy	no-referrer-when-downgrade
▼ Response Headers	
Access-Control-Allow-Origin	*
Cache-Control	max-age=300
Cross-Origin-Resource-Policy	cross-origin
Date	Fri, 24 Oct 2025 14:38:56 GMT
Etag	W/"90f9c0f1663fa52846bcbe86835fb5a3346653e67a26c6fee0fe2635c357e290"
Expires	Fri, 24 Oct 2025 14:43:56 GMT
Vary	Authorization, Accept-Encoding
▼ Request Headers	
:authority	raw.githubusercontent.com
:method	GET
:path	/gin-gonic/gin/refs/heads/master/AUTHORS.md
:scheme	https
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	en-US,en;q=0.9
Cache-Control	max-age=0
If-None-Match	W/"90f9c0f1663fa52846bcbe86835fb5a3346653e67a26c6fee0fe2635c357e290"

GET <https://raw.githubusercontent.com/gin-gonic/gin/refs/heads/master/AUTHORS.md>

Try hard-reload (Ctrl+F5) ● 200 OK

REST - 6 Guiding Principles

- Client-Server

- Separation of concerns
- Modularization
- Features:
 - Portability of client
 - Scalability of server

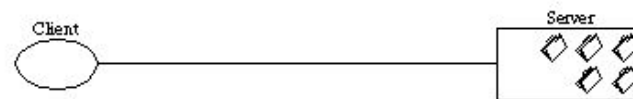


Figure S-2. Client-Server

- **Stateless**
- Cacheable
- Uniform Interface
- Layered System
- Code on Demand (*)

- Recall Lab4 when we intend to refactor a local program into a modular service.

```
my_service.py x
my_service.py > AssistantService > greet_with_info
1 # A Modular Assistant Service.
2 class AssistantService:
3     def __init__(self) -> None:
4         # Add some properties.
5         pass
6
7     # Greet the user with provided user name and institution.
8     def greet_with_info(self, username, institution):
9         | return f'Hello {username} from {institution}!'
10
11     # Multiply two numbers.
12     def mult(self, xin, yin):
13         | return xin * yin
14
```

```
my_client.py x
my_client.py > ...
1 from my_service import AssistantService
2
3 # Client to use the Procedure
4 if __name__ == '__main__':
5     svc = AssistantService()
6     print(svc.greet_with_info(username='Peter', institution='SUSTech'))
7     print(svc.mult(xin=3.5, yin=5))
8
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(dncc) (base) root@RAINBOW:~/rainbow/asialab/dncc/local_service# python my_client.py
Hello Peter from SUSTech!
17.5
```

REST - 6 Guiding Principles

- Client-Server
- **Stateless**
 - No session state on server
 - Request contains all necessary info
 - Requests do not rely on one another
 - Features:
 - Visibility
 - Reliability
 - Scalability
 - Concern: repetitive data overhead
- Cacheable
- Uniform Interface
- Layered System
- Code on Demand (*)

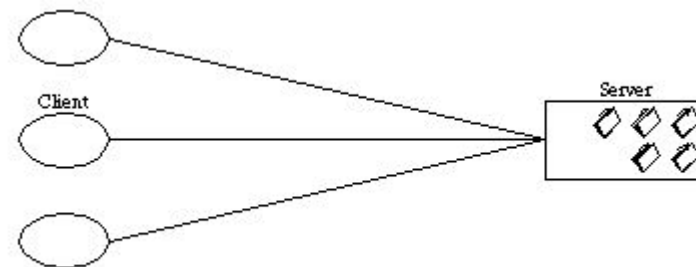


Figure 5-3. Client-Stateless-Server

Request:

1. Add a new book (id=1) in the bookshelf.
2. **Give me that created book.**



Request:

1. Add a new book (id=1) in the bookshelf.
2. **Give me the book (id=1).**



REST - 6 Guiding Principles

- Client-Server
- **Stateless**
- **Cacheable**
 - Response data should implicitly or explicitly announce cacheability.
 - If cacheable, client cache is reused.
 - Features:
 - Efficiency
 - ...
 - Concern: stale data handling
- Uniform Interface
- Layered System
- Code on Demand (*)

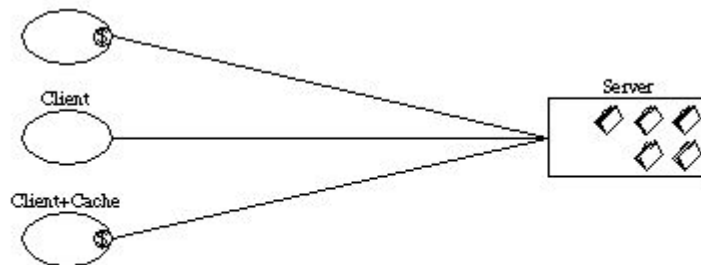


Figure 5-4. Client-Cache-Stateless-Server

Response:

1. The story is: "Alice was beginning to ..."
2. **The story does not change. Check the previous copy.**

● 200 OK

● 304 Not Modified

Etag:	W/"25-Xqwo/6dp/X3uWqc0VJHdsdRW3Xg"
Cache-Control:	max-age=0
If-None-Match:	W/"25-Xqwo/6dp/X3uWqc0VJHdsdRW3Xg"

REST - 6 Guiding Principles

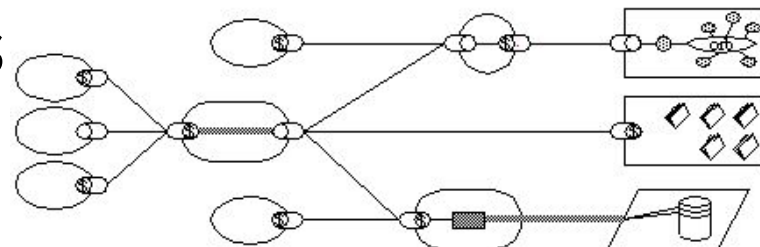
- Client-Server
- **Stateless**
- Cacheable
- **Uniform Interface**
 - Principle of generality
 - **Identification of resources**: uniquely identify each resource
 - **Manipulation of resources through representations**: uniform representations
 - **Self-descriptive messages**: enough info in each resource representation
 - **Hypermedia as the engine of application state**: drive resources via hyperlinks
 - Features:
 - Simplicity
 - Independent evolvability
 - ...
 - Concern: degradation of efficiency (general vs. optimized)
- Layered System
- Code on Demand (*)

```
{
  "id": 123,
  "title": "What is REST",
  "content": "REST is an architectural style for building web services...",
  "published_at": "2023-11-04T14:30:00Z",
  "author": {
    "id": 456,
    "name": "John Doe",
    "profile_url": "https://example.com/authors/456"
  },
  "comments": {
    "count": 5,
    "comments_url": "https://example.com/posts/123/comments"
  },
  "self": {
    "link": "https://example.com/posts/123"
  }
}
```

NOTE: Principles are just guidelines. It is not compulsory to follow all mentioned guidelines to become a RESTful API.

REST - 6 Guiding Principles

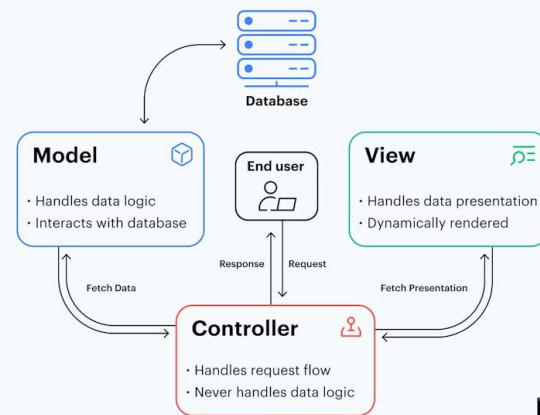
- Client-Server
- **Stateless**
- Cacheable
- Uniform Interface
- **Layered System**
 - Hierarchical layers
 - Example: MVC pattern, **Reverse Proxy** (next week)
 - Features:
 - Reduced coupling
 - Evolvability
 - Reusability
 - Scalability
 - Transparency
 - Concern: complexity & processing overhead
- Code on Demand (*)



Client Connector: Client+Cache: Server Connector: Server+Cache:

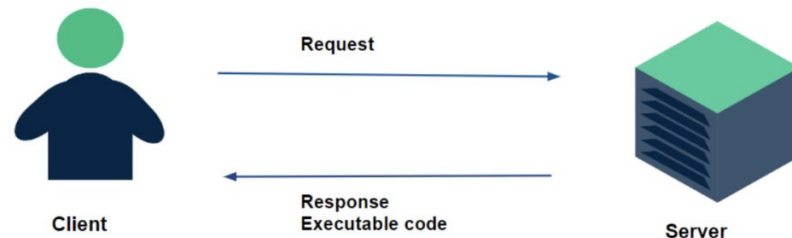
Figure 5-7. Uniform-Layered-Client-Cache-Stateless-Server

MVC Architecture



REST - 6 Guiding Principles

- Client-Server
- **Stateless**
- Cacheable
- Uniform Interface
- Layered System
- **Code on Demand (*)**
 - Extend client functionality by downloading code from server
 - Example: server sends JavaScript code to client to execute
 - Features:
 - Simplicity of client init feature
 - Flexibility
 - Concern: visibility & security



RESTful API Components

Take OpenAPI specification as an example:

- Resource: identified by a URL endpoint.
- HTTP method: specifies an operation on the resource (e.g., GET, POST, etc.).
- HTTP request:
 - Header: metadata
 - Content-Type = application/json
 - Query Parameters
 - /users?age=25&sort=desc
 - Request Body
- HTTP response:
 - Status code + message
 - 200 OK
 - 404 Not Found
 - ...
 - Header
 - Response Body

Request URL
https://petstore3.swagger.io/api/v3/pet

POST /pet Add a new pet to the store

Add a new pet to the store

Parameters Try it out

No parameters

Request body required application/json

Create a new pet in the store

Example Value | Schema

```
{
  "id": 10,
  "name": "doggie",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

Responses HTTP response

Code	Description	Links
200	Successful operation	No links

Media type
application/json

Controls Accept Header:

Example Value | Schema

RESTful API - Benefits

- **Scalability**

- a. Client-Server enables multiple server duplicates with load balancing.
- b. Stateless ensures that each request holds the entire context and is independently processable, thus the servers can be easily scaled out.
- c. Cacheable reduces the load of the servers so they can server more clients. Introducing distributed caching further improves the service capacity.

- **Flexibility & Independence**

- a. Client-Server allows the client and the server to be evolved independently without affecting each other.
- b. Layered System further decouples service logic so that each layer can be designed and implemented separately.
- c. Flexibility: Code on Demand allows the client to extend new functionalities when needed, provided by the server via downloading code.

- Everything is organized thanks to Uniform Interface.

RESTful API vs. gRPC

	 gRPC	 REST
Data format	Uses Protobuf to encode data in binary form	Uses plain-text data formats (JSON and XML)
Data validation	Automatically validates every message against the API contract	Requires an extra validation step on JSON data
Communication pattern	Supports unary communication, as well as server streaming, client streaming, and bidirectional streaming	Follows a unary request/response cycle
Design pattern	Defines callable functions on the server, which the client can invoke as if they were local	Uses HTTP methods to grant access to resources through dedicated endpoints
Code generation	Supports code generation in many programming languages	No native support for code generation
Primary use case	Microservice architectures	Public APIs or other APIs where ease of use is a priority



TASK: RESTful API - Hello World with Python Flask

Implement a simple RESTful API server using Python Flask.

> Reference codebase: [rest_hello_world](#)

1. Set up Python ([uv](#) is recommended).
2. Install Python dependencies into a uv environment via:
 - `uv sync`
3. Run the API server via:
 - `uv run python server.py`
4. In another terminal, test the API with HTTP requests.

```
21 @app.route('/', methods=['GET'])
22 def greet():
23     return {'message': 'Hello World!'}, 200
24
25 @app.route('/chat/<username>', methods=['GET'])
26 def greet_with_info(username): # retrieve username from URL path
27     # retrieve institution from URL query
28     institution = request.args.get('institution', None)
29     institution_segment = f' from {institution}' if institution else ''
30     msg = f'Hello {username}{institution_segment}!'
31     return {'message': msg}, 200
32
33 @app.route('/calculator/mult', methods=['POST'])
34 def mult():
35     inputs = request.get_json()
36     op = MultOp(xin=inputs['xin'], yin=inputs['yin'])
37     op.cal()
38     return op.to_json(), 200
```

```
root@RAINBOW:~/rainbow/asialab/dccc/dccc-lab/rpc_rest/1_rest/8_hello_world/hello_world# uv run python server.py
```

```
* Serving Flask app 'server'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production
* Running on http://127.0.0.1:8081
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 846-852-296
```

```
root@RAINBOW: # curl http://localhost:8081/
{
  "message": "Hello World!"
}
root@RAINBOW: # curl http://localhost:8081/chat/Peter?institution=SUSTech
{
  "message": "Hello Peter from SUSTech!"
}
root@RAINBOW: # curl -X POST -H "Content-Type: application/json" -d '{"xin": 1.5,
"yin": 6}' http://localhost:8081/calculator/mult
{
  "result": 9.0,
  "xin": 1.5,
  "yin": 6
}
```


OpenAPI As a RESTful API Specification

- Refer to [OpenAPI Specification from Swagger](#).
- Specification document file can be either JSON or **YAML**.
- Metadata:
 - OpenAPI version
 - Info:
 - Title
 - API spec version
 - Tags (*) for operations
- API paths & operations
 - URL endpoint
 - Method
 - Request & Response
- Components - Schemas
 - Set of reusable data objects

```
! hello_world.yaml X
rpc_rest > 1_rest > 0_hello_world > ! hello_world.yaml > {} info
OpenAPI 3.0.X (v3.json) | Scan | Audit

1  openapi: 3.0.3
2  info:
3    title: Hello World
4    description: A basic example showing how to write basic RESTful APIs.
5    version: 1.0.0 # defined by yourself
6
7  # Tags cluster API operations into different groups for fine-grained control.
8  tags:
9    - name: Dialog...
10
11   - name: Calculator...
12
13
14  # Paths define API endpoints and operations.
15  paths:
16    /: ...
17
18   /chat/{username}: ...
19
20   /calculator/mult: ...
21
22
23
24  # Components define reusable data objects to be used in API operation definitions.
25  components:
26    schemas:
27      # An multiplying operation.
28      MultOp: ...
29
30      ### Request Body: it is a good habit to separately define a request body object for each request,
31      ### rather than using the same resource object (e.g., MultOp).
32      MultRequestBody: ...
33
34
35      ### Response: it is a good habit to pre-define reusable responses.
36      responses:
37        DialogResponse: ...
```

metadata

APIs

data object schema

OpenAPI As a RESTful API Specification

A GET example

- Refer to [OpenAPI Specification from Swagger](#).
- Specification document file can be either JSON or **YAML**.
- Metadata:
 - OpenAPI version
 - Info:
 - Title
 - API spec version
 - Tags (*) for operations
- API paths & operations
 - URL endpoint
 - Method
 - Request & Response
- Components - Schemas
 - Set of reusable data objects

```
! hello_world.yaml x
rpc_rest > 1_rest > 0_hello_world > ! hello_world.yaml > {} components > {} schemas
15  paths:
28    /chat/{username}: URL endpoint
29    get:
HTTP GET Scan | Try it | Audit
30    tags:
31      - Dialog
32    description: Get a greeting with provided name and institution.
33    operationId: greet_with_info generated function name in the backend server
34    parameters:
35      # this parameter is in the URL path
36      - name: username
37        in: path
38        required: true a path parameter
39        schema:
40          type: string
41          example: 'peter'
42      # this parameter is in the URL query (e.g., /chat/peter?institution=SUSTech)
43      - name: institution
44        in: query
45        required: false a query parameter
46        schema:
47          type: string
48          example: 'SUSTech'
49      responses:
50        '200': response status code
51          $ref: '#/components/responses/DialogResponse'
```

```
75  components:
110    responses:
111      DialogResponse:
112        description: Successful Dialog Response
113        content:
114          application/json:
115            schema:
116              type: object
117              required:
118                - message
119              properties:
120                message:
121                  type: string
```

OpenAPI As a RESTful API Specification

A POST example

- Refer to [OpenAPI Specification from Swagger](#).
- Specification document file can be either JSON or **YAML**.
- Metadata:
 - OpenAPI version
 - Info:
 - Title
 - API spec version
 - Tags (*) for operations
- API paths & operations
 - URL endpoint
 - Method
 - Request & Response
- Components - Schemas
 - Set of reusable data objects

```
! hello_world.yaml X
rpc_rest > 1_rest > 0_hello_world > ! hello_world.yaml > {} paths > {} /calculator/mult

15  paths:
53    /calculator/mult: → URL endpoint
54      post:
55        HTTP POST Scan | Try it | Audit
56        tags:
57          - Calculator
58        description: Multiply two numbers.
59        operationId: mult → generated function name
60        requestBody:      in the backend server
61          required: true
62          content:
63            # a JSON type request body
64            application/json: → request body media type
65              schema:
66                reference to a data object ← $ref: '#/components/schemas/MultRequestBody'
67              responses:
68                '200':
69                  description: Successful operation
70                  content: → response body media type
71                    application/json:
72                      schema:
73                        reference to a data object ← $ref: '#/components/schemas/MultOp'
```

```
75  components:
76    schemas:
77      # An multiplying
78      MultOp:
79        type: object
80        required: ...
81        properties:
82          xin: ...
83          yin: ...
84          result: ...
85
86      ### Request Body:
87      ### rather than I
88      MultRequestBody:
89        type: object
90        required: ...
91        properties:
92          xin: ...
93          yin: ...
```

OpenAPI - Benefits

- **Standardization**
 - Client & Server reaches an agreement on available APIs and access requirements.
- **Human-Readable Format**
 - JSON/YAML is both clean & programmatically operable.
- **API Documentation & Visualization:**
 - [Swagger UI](#): visualize & interact with APIs in a web page
 - [Swagger Editor](#): edit API specification file with real-time Swagger UI support
- **Multi-Language Client & Server Code Template Generation:**
 - Client/Server code of different frameworks from different programming languages can be automatically generated as development templates.
 - [OpenAPI Generator](#); [oapi-codegen](#) (go-specific)
- **Integrated Testing Capability:**
 - Variable tools support easy setup for API testing.
 - [cURL](#)
 - [Swagger Mock Server](#)
 - [Postman](#)

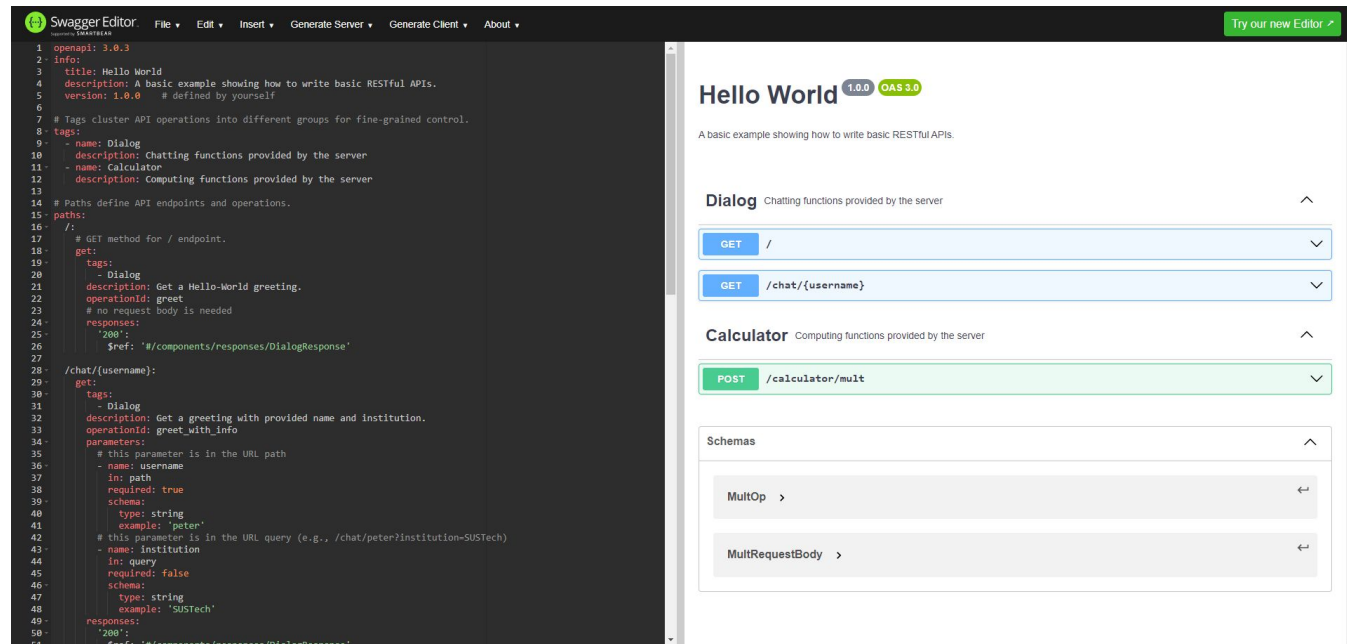
TASK: RESTful API - Hello World with Python Flask

Implement a simple RESTful API server using Python Flask.

> Reference codebase: [rest_hello_world](#)

5. Check the API specification file [hello_world.yaml](#)

6. Copy the file content into the [online Swagger Editor](#). Explore the generated Swagger UI page.



The image shows the Swagger Editor interface. On the left, the OpenAPI specification is displayed in a code editor. On the right, the generated Swagger UI is shown, featuring a 'Hello World' title, a description, and two API endpoints: 'Dialog' and 'Calculator'.

OpenAPI Specification (Left Panel):

```

1 openapi: 3.0.3
2 info:
3   title: Hello World
4   description: A basic example showing how to write basic RESTful APIs.
5   version: 1.0.0 # defined by yourself
6
7 # Tags cluster API operations into different groups for fine-grained control.
8 tags:
9   - name: Dialog
10     description: Chatting functions provided by the server
11   - name: Calculator
12     description: Computing functions provided by the server
13
14 # Paths define API endpoints and operations.
15 paths:
16   /:
17     # GET method for / endpoint.
18     get:
19       tags:
20         - Dialog
21       description: Get a Hello-World greeting.
22       operationId: greet
23       # no request body is needed
24       responses:
25         '200':
26           $ref: '#/components/responses/DialogResponse'
27
28   /chat/{username}:
29     get:
30       tags:
31         - Dialog
32       description: Get a greeting with provided name and institution.
33       operationId: greet_with_info
34       parameters:
35         # this parameter is in the URL path
36         - name: username
37           in: path
38           required: true
39           schema:
40             type: string
41             example: 'peter'
42         # this parameter is in the URL query (e.g., /chat/peter?institution=SUSTech)
43         - name: institution
44           in: query
45           required: false
46           schema:
47             type: string
48             example: 'SUSTech'
49       responses:
50         '200':
51           $ref: '#/components/responses/DialogResponse'
  
```

Swagger UI (Right Panel):

Hello World 1.0.0 OAS 3.0

A basic example showing how to write basic RESTful APIs.

Dialog Chatting functions provided by the server

- GET /
- GET /chat/{username}

Calculator Computing functions provided by the server

- POST /calculator/mult

Schemas

- MultiOp >
- MultiRequestBody >

Another RESTful API Example - Petstore


Swagger Editor
File Edit Insert Generate Server Generate Client About

```

1 openapi: 3.0.3
2 info:
3   title: Swagger Petstore
4   description: |-
5     This is a sample Pet Store Server based on the
6     Swagger at [https://swagger.io]. In the third iteration of the pet store, we've switched
7     to the design first approach. You can now help us improve the API by making changes to the definition itself or to the code.
8     That way, with time, we can improve the API in general, and expose some of the new features in OAS3.
9
10    _If you're looking for the Swagger 2.0/OAS 2.0 version of Petstore, then click [here](https://editor.swagger.io/?url=https://petstore.swagger.io/v2/swagger.yaml). Alternatively, you can load via the 'Edit > Load
11    Petstore OAS 2.0' menu option!_
12
13    Some useful links:
14    - [The Pet Store repository](https://github.com/swagger-api/swagger-petstore)
15    - [The source API definition for the Pet Store](https://github.com/swagger-api/swagger-petstore/blob/master/src/main/resources/openapi.yaml)
16
17    termsOfService: http://swagger.io/terms/
18    contact:
19      email: apiteam@swagger.io
20
21    license:
22      name: Apache 2.0
23      url: http://www.apache.org/licenses/LICENSE-2.0.html
24
25    version: 1.0.11
26
27  externalDocs:
28    description: Find out more about Swagger
29    url: http://swagger.io
30
31  servers:
32    - url: https://petstore3.swagger.io/api/v3
33
34  tags:
35    - name: pet
36      description: Everything about your Pets
37      externalDocs:
38        description: Find out more
39        url: http://swagger.io
40
41    - name: store
42      description: Access to Petstore orders
43      externalDocs:
44        description: Find out more about our store
45        url: http://swagger.io
46
47    - name: user
48      description: Operations about user
49
50  paths:
51    /pet:
52      put:
53        tags:
54          - pet
55        summary: Update an existing pet
56        description: Update an existing pet by Id
57        operationId: updatePet

```

Convert to YAML
Load Petstore OAS 3.0
Load Petstore OAS 2.0

Swagger Petstore - OpenAPI 3.0 1.0.11 OAS 3.0

This is a sample Pet Store Server based on the OpenAPI 3.0 specification. You can find out more about Swagger at <https://swagger.io>. In the third iteration of the pet store, we've switched to the design first approach! You can now help us improve the API whether it's by making changes to the definition itself or to the code. That way, with time, we can improve the API in general, and expose some of the new features in OAS3.

If you're looking for the Swagger 2.0/OAS 2.0 version of Petstore, then click [here](#). Alternatively, you can load via the [Edit > Load Petstore OAS 2.0](#) menu option!

Some useful links:

- [The Pet Store repository](#)
- [The source API definition for the Pet Store](#)

Terms of service

Contact the developer

Apache 2.0

Find out more about Swagger

Servers

<https://petstore3.swagger.io/api/v3>

[Authorize](#)

pet Everything about your Pets [Find out more](#)

PUT [/pet](#) Update an existing pet

POST [/pet](#) Add a new pet to the store

GET [/pet/findByStatus](#) Finds Pets by status

Another RESTful API Example - Petstore

The image shows the Swagger Editor on the left and the Swagger UI on the right. The Swagger Editor displays the OpenAPI 3.0 specification for a Pet Store API. The Swagger UI shows the details for the GET /pet/findByStatus endpoint, including the status parameter, the Execute button, and the response body. A red box highlights the 'Execute' button and the 'Responses' section. Red arrows point to the 'status' parameter, the 'Execute' button, and the 'Response body' section.

Swagger JSON Definition (Left Pane):

```

openapi: 3.0.3
info:
  title: Swagger Petstore - OpenAPI 3.0
  description: |-
    This is a sample Pet Store Server based on the OpenAPI 3.0 specification. You can find out more about
    Swagger at [https://swagger.io](https://swagger.io). In the third iteration of the pet store, we've switched
    to the design first approach!
    You can now help us improve the API whether it's by making changes to the definition itself or to the code.
    That way, with time, we can improve the API in general, and expose some of the new features in OAS3.
  _If you're looking for the Swagger 2.0/OAS 2.0 version of Petstore, then click [here](https://editor.swagger
  .io/?url=https://petstore.swagger.io/v2/swagger.yaml). Alternatively, you can load via the 'Edit > Load
  Petstore OAS 2.0' menu option!_
  Some useful links:
  - [The Pet Store repository](https://github.com/swagger-api/swagger-petstore)
  - [The source API definition for the Pet Store](https://github.com/swagger-api/swagger-petstore/blob/master
  /src/main/resources/openapi.yaml)
  termsOfService: http://swagger.io/terms/
  contact:
    email: apiteam@swagger.io
  license:
    name: Apache 2.0
    url: http://www.apache.org/licenses/LICENSE-2.0.html
  version: 1.0.11
externalDocs:
  description: Find out more about Swagger
  url: http://swagger.io
servers:
  - url: https://petstore3.swagger.io/api/v3
tags:
  - name: pet
    description: Everything about your Pets
    externalDocs:
      description: Find out more
      url: http://swagger.io
  - name: store
    description: Access to Petstore orders
    externalDocs:
      description: Find out more about our store
      url: http://swagger.io
  - name: user
    description: Operations about user
paths:
  /pet:
    put:
      tags:
        - pet
      summary: Update an existing pet
      description: Update an existing pet by Id
      operationId: updatePet
      requestBody:
        description: Update an existent pet in the store
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Pet'
          application/xml:
            schema:
              $ref: '#/components/schemas/Pet'

```

Swagger UI (Right Pane):

GET /pet/findByStatus Finds Pets by status

Multiple status values can be provided with comma separated strings

Parameters

Name	Description
status	Status values that need to be considered for filter

string

(query)

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/findByStatus?status=available' \
  -H 'accept: application/json'
```

Request URL

```
https://petstore3.swagger.io/api/v3/pet/findByStatus/status=available
```

Server response

Code Details

Response from the Server

200

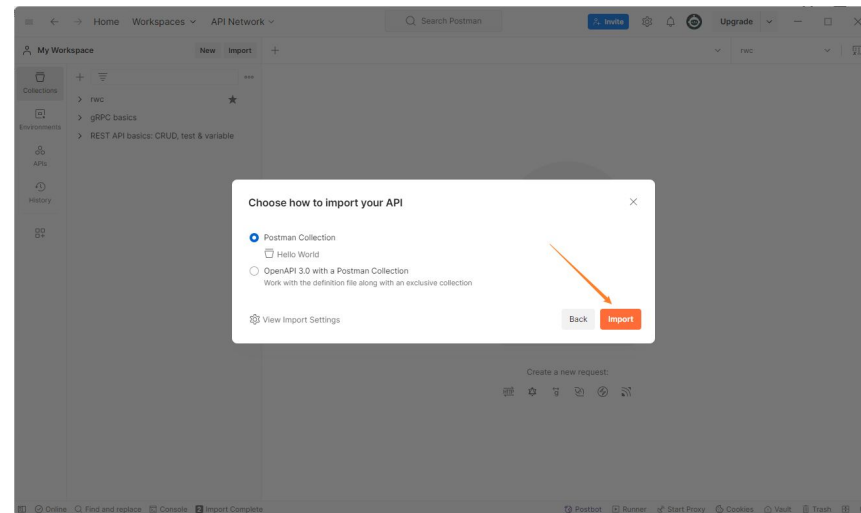
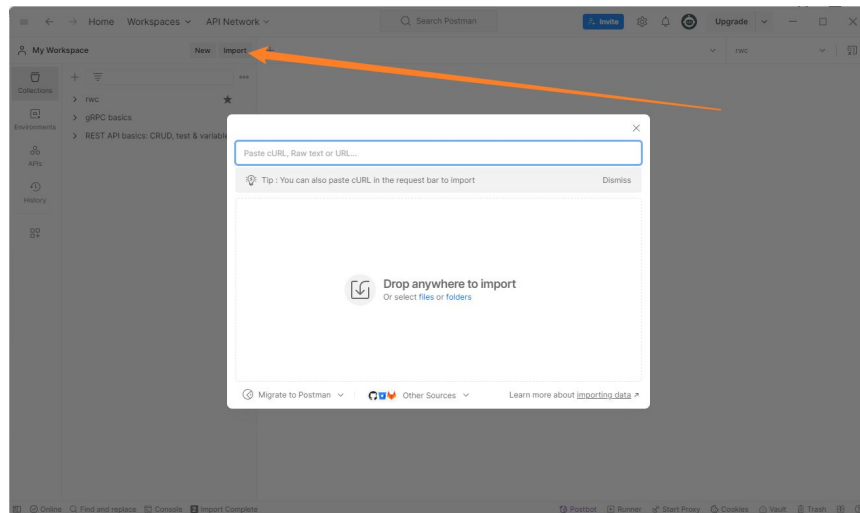
Response body

```
{
  "id": 7759440800036640000,
  "category": {
    "id": 3390426472185137000,
    "name": "MAVRZVL'y0B'ZcUY6-{FNO-VJ2}3uF;ar3ER_x+y))('",
    "name": "y27M1B0as5vAN(N''*o=H[1g02;0L-vy,5{1,1'",
    "photoUrls": [
      "Lzm3j'KYpF/!ot[1m03S'b>60t<u)-X,t-DF;KL3[(-X)AP][('K"
    ],
    "tags": [

```

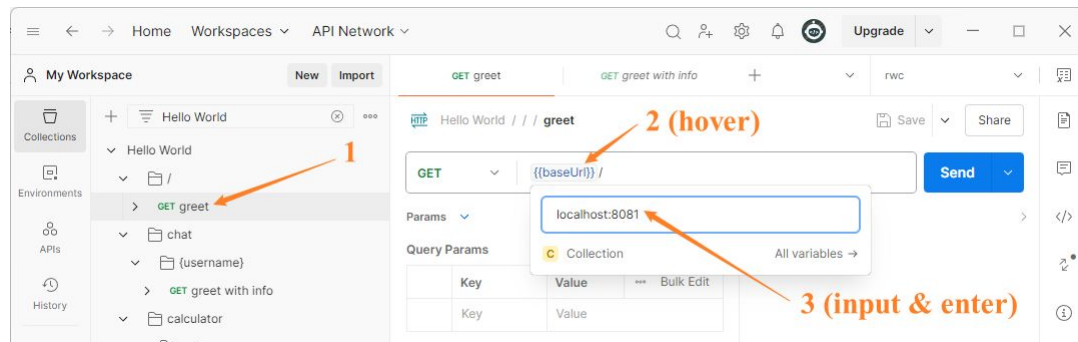
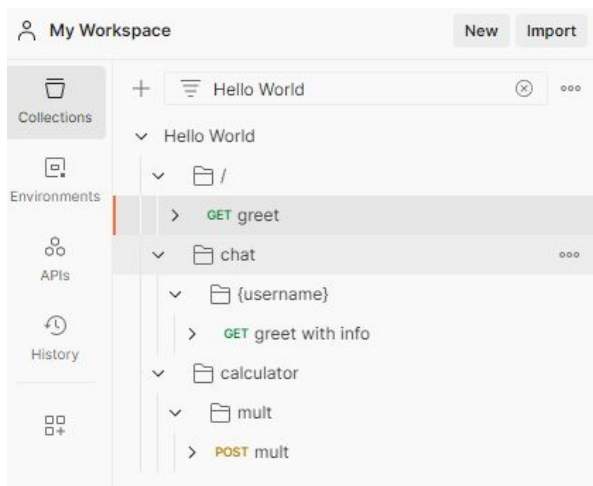
Postman for RESTful API Testing

1. Download and install [Postman Desktop app](#).
2. In the workspace, click the “Import” button.
3. Upload the API specification YAML file (e.g., `hello_world.yaml`). Click “Import”.



Postman for RESTful API Testing

4. A collection of API testing requests will be automatically generated. Note that the URL endpoint contains a base url variable that needs to be configured.
5. Select any request in the collection, hover on the `{{baseUrl}}` variable in the URL endpoint and change its value to the address of the running API backend server.



Postman for RESTful API Testing

6. Now play with these requests with different parameters/body.

The screenshot displays the Postman interface with a REST client request configured. The request is a GET method to the endpoint `{{baseUrl}}/chat/username?institution=SUSTech`. The Params tab shows two query parameters: `Key` and `institution` (value: `SUSTech`). The Path Variables tab shows `username` (value: `Peter S`, description: `(Required)`). The response is a 200 OK status with a JSON body: `{ "message": "Hello Peter S from SUSTech!" }`.

Annotations in the image:

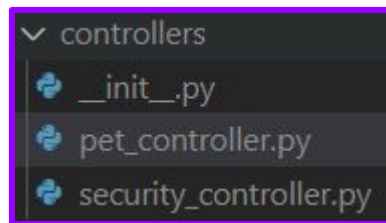
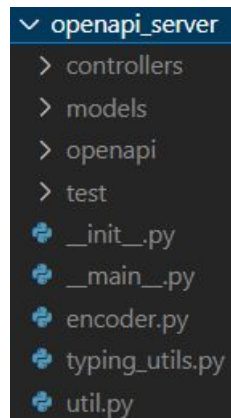
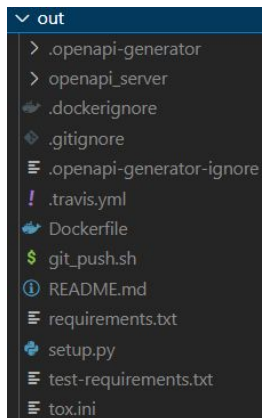
- 1: Points to the `GET greet with info` request in the sidebar.
- 2: Points to the `institution` query parameter in the Params tab.
- 3: Points to the `Send` button.
- 4 (response): Points to the JSON response body.

TASK: RESTful API via Codegen

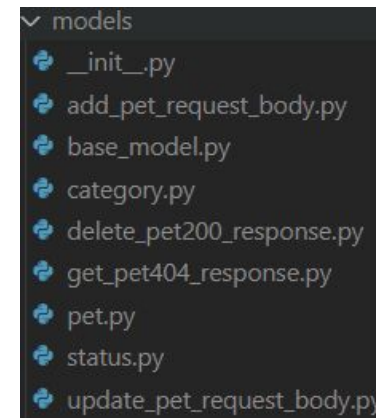
Generate Flask server code template via OpenAPI Generator.

> Reference codebase: [rest_codegen](#)

1. Set up Python ([uv](#) is recommended).
2. Check the API specification file `petstore.yaml`. Using [Docker + OpenAPI Generator](#), generate Python Flask server code template with the OpenAPI Generator CLI:
 - `docker run --rm -v ./:/app/ openapitools/openapi-generator-cli generate -i /app/petstore.yaml -g python-flask -o /app/out/`
3. The server template includes data object models and API controllers.



Need to implement



TASK: RESTful API via Codegen

Generate Flask server code template via OpenAPI Generator.

> Reference codebase: [rest_codegen](#)

4. Implement the Pet Controller. Our sample implementation follows the MVSC pattern (Model-View-Service-Controller).

```
pet_controller.py M X
rpc_rest > 1_rest > 1_codegen > generated > openapi_server > controllers > pet_controller.py > ...

15 pet_svc = PetService()
16
17 def update_pet(pet_id): # noqa: E501
18     """update_pet ...
19
20     if connexion.request.is_json:
21         update_pet_request_body = UpdatePetRequestBody.from_dict(connexion.request.get_json())
22         pet = pet_svc.update_pet(pet_id, name=update_pet_request_body.name,
23                                 category=update_pet_request_body.category,
24                                 status=update_pet_request_body.status)
25
26     if pet is None:
27         return {"message": "no pet with id=%s" % pet_id}, 404
28     else:
29         return pet, 200
```

Annotations:

- `pet_svc = PetService()`: a Pet Service in a lower layer
- `update_pet(pet_id)`: matches op id in API spec
- `pet_id`: a path parameter
- `connexion.request.get_json()`: decode request body
- `UpdatePetRequestBody.from_dict()`: deserialize request body
- `pet_svc.update_pet()`: handle request via Pet Service
- `return pet, 200`: return response body, status code

TASK: RESTful API via Codegen

Generate Flask server code template via OpenAPI Generator.

> Reference codebase: [rest_codegen](#)

5. Configure Python dependencies:
 - `cd out/; uv init; uv add -r requirements.txt`
6. Run the server as a Python module:
 - `cd out/; uv run python -m openapi_server`
7. In another terminal, test the API with HTTP requests.

```
python -m openapi_server
* Serving Flask app '__main__' (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a p
roduction WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:8080
* Running on http://172.30.236.136:8080
Press CTRL+C to quit
```



```
(base) root@RAINBOW: # curl http://127.0.0.1:8080/pets
[]
(base) root@RAINBOW: # curl -X POST http://127.0.0.1:8080/pets \
-H "Content-Type: application/json" \
-d '{"name": "Bond", "category": {"id": 1, "name": "Dogs"}, "status": "healthy"}'
{
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "id": 1,
  "name": "Bond",
  "status": "healthy"
}
(base) root@RAINBOW: # curl http://127.0.0.1:8080/pets
[
  {
    "category": {
      "id": 1,
      "name": "Dogs"
    },
    "id": 1,
    "name": "Bond",
    "status": "healthy"
  }
]
```

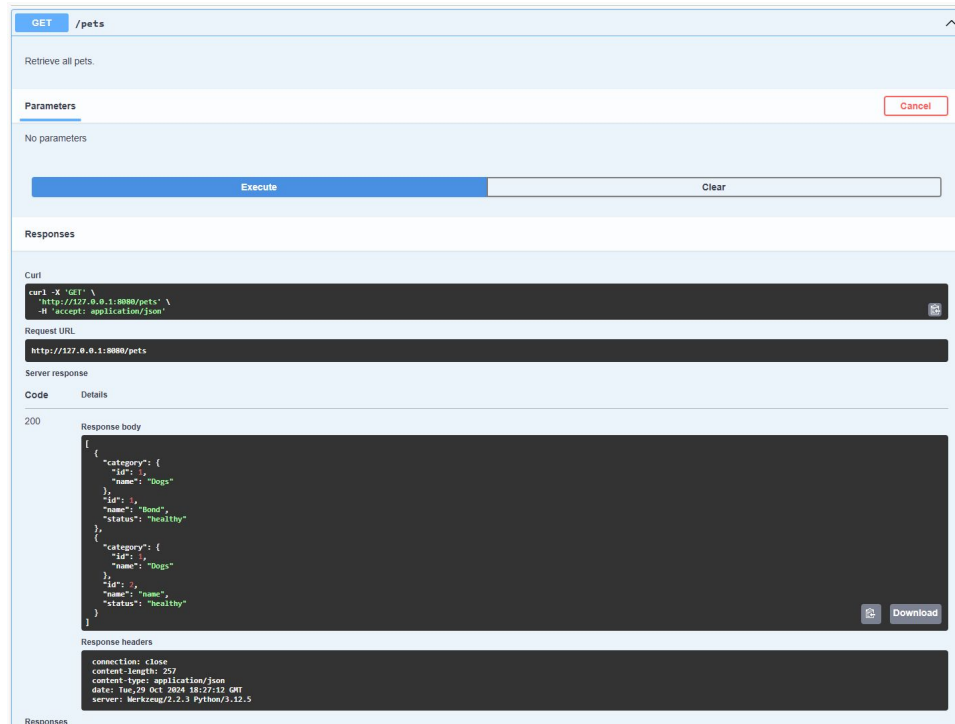
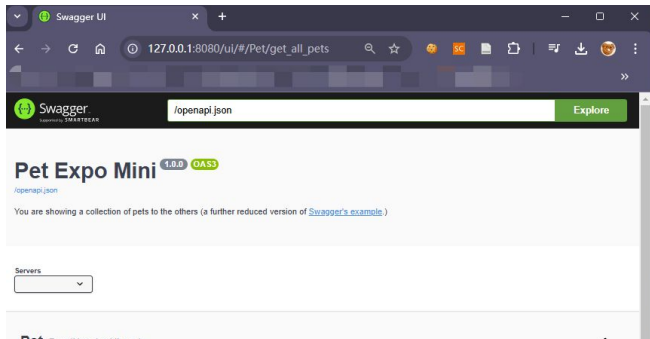
TASK: RESTful API via Codegen

This task explores all the aforementioned benefits.

Generate Flask server code template via OpenAPI Generator.

> Reference codebase: [rest_codegen](#)

- The generator [naturally supports Swagger UI](#). The UI web page is serving at the `<BASE_URL>/ui` endpoint. In this task, access <http://127.0.0.1:8080/ui> in the browser.
- All test requests invoked from the UI page will be forwarded to the API server.



TASK: RESTful API - Codegen Yourself

Implement a RESTful API server yourself with OpenAPI Generator.

> Reference codebase: [rest_codegen_yourself](#)

Write an addition function to add an array of integers together and return the result. Define a RESTful API with the calculator tag to handle this functionality. Add this API to `hello_world.yaml`.

1. Modify API specification YAML file.
2. Generate via OpenAPI Generator.
3. Implement the generated controllers.
4. Test the new API.

Try starting from scratch to be familiar with the process before Assignment 2.

The screenshot displays a REST client interface for a POST request to the endpoint `/calculator/add`. The description states "Add an array of numbers." The "Parameters" section indicates "No parameters". The "Request body" is set to "application/json" and is required. An example value is shown as `{ "vals": [1,2,3,4,5,6,7,8,9,10] }`. The "Responses" section shows a 200 status code. A cURL command is provided: `curl -X 'POST' \ 'http://localhost:8080/calculator/add' \ -H 'accept: application/json' \ -H 'Content-Type: application/json' \ -d '{ "vals": [1,2,3,4,5,6,7,8,9,10] }'`. The "Request URL" is `http://localhost:8080/calculator/add`. The "Server response" shows a 200 status code with a response body of `{ "result": 55 }`.

Summary

- **REST**

- a. Components: resource, representation, control data
- b. 6 Guiding Principles
 - i. Client-Server
 - ii. **Stateless**
 - iii. Cacheable
 - iv. Uniform Interface
 - v. Layered System
 - vi. Code on Demand (Optional)

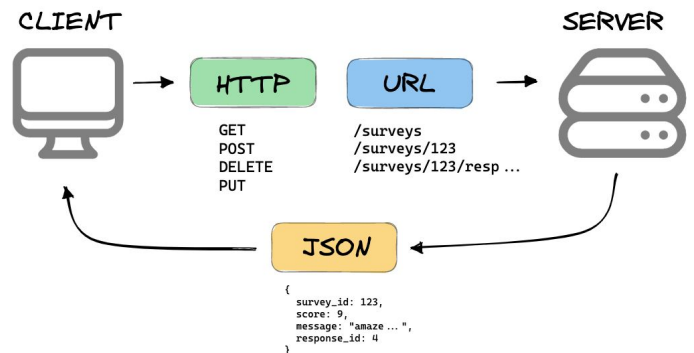
- **RESTful API**

- a. Benefits: Scalability, Flexibility, Independence
- b. RESTful API vs. gRPC

- **OpenAPI**

- a. Components: metadata, API paths & operations, data object schemas
- b. Benefits: Standardization, Human-readable, UI, Codegen, Integrated Testing

WHAT IS A REST API?



mannhowie.com